QALight

# Frontend-розробка

## Заняття 8. Типи позиціонування. Форми

Викладач курсу: Іван Ніконов, ТОВ «Тренінг-центр КьюЕйЛайт»

# Порядок денний



Статичне, відносне, абсолютне та фіксоване позиціонування. Координати, зміна точки початку координат, відносні координати та розміри, координати за умовчанням.
Елемент form. Інтерактивні елементи. Атрибути тега input. Мітки елементів. Опис елементів. Перемикачі.

Створення абсолютно позіціонованої форми контактів.

## Типи позиціонування

Статичне, відносне, абсолютне та фіксоване позиціонування

# Властивість position

Теорія: Властивість position

Властивість CSS **position** вказує, як елемент позиціонується у документі. **top, right, bottom та left** визначають місцезнаходження позиціонованих елементів.

Існують такі значення position **relative, absolute, fixed, або sticky.**

**Відносна** позиція елемента задається значенням relative. Властивості top та bottom визначають розміщення по вертикалі від нормального положення; властивості left і right задають горизонтальне розміщення.

**Абсолютне** позиціонування елемента задається значенням **absolute або fixed**. top, right, bottom та left задають відстані від границь батьківського блоку елемента. Якщо елемент має поля, вони додаються до відстаней.

Елемент з **липкою** позицією - це елемент, який має значення position sticky. Елемент вважається відносно позиційованим до тих пір, поки його батьківський блок не перетне вказаний поріг всередині кореневого потоку, після чого обробляється як абсолютно позиційованим.

# Статичне позиціонування

Теорія: позиціонування

Елементи за замовчуванням мають значення положення **static**, тобто вони не мають і не приймають жодних конкретних властивостей зміщення. Крім того, елементи будуть розташовуватися так, як задумано, зі своєю поведінкою за замовчуванням.

```
<div class="box-set">
 <figure class="box box-1">Box 1</figure>
 <figure class="box box-2">Box 2</figure>
 <figure class="box box-3">Box 3</figure>
 <figure class="box box-4">Box 4</figure>
</div>
```

# Відносне позиціонування

Теорія: позиціонування

Відносне значення властивості **position: relative** дуже схоже на значення статичного значення. Основна відмінність полягає в тому, що відносне значення приймає властивості зміщення поля зверху, справа, знизу та зліва. Ці властивості зміщення дозволяють елементу бути точно позиціонованим, зміщуючи елемент із положення за замовчуванням у будь-якому напрямку.

Властивості зміщення поля, зверху, справа, знизу та зліва, визначають, як елементи можуть бути розміщені та в якому напрямку. Ці властивості зміщення працюють лише для елементів із відносним, абсолютним або фіксованим значенням позиціонування.

Для відносно розташованих елементів ці властивості визначають, як елемент слід перемістити з його положення за замовчуванням. Наприклад, використання верхнього значення 20 пікселів на відносно розташованому елементі відсуне елемент на 20 пікселів вниз від того місця, де він був спочатку розміщений. Перемикання верхнього значення на -20px натомість витягне елемент на 20 пікселів угору від того місця, де він був спочатку розміщений.

# Відстан  між елементом

Теорія: позиціонування

Для елементів з абсолютним або фіксованим позиціонуванням властивість **position** визначає **відстань між елементом і краями його батьківського елемента**. Наприклад, використання значення top: 20px; для абсолютно розташованого елемента відсуне елемент на 20 пікселів вниз від вершини його відносно розташованого батька. Перемикання верхнього значення на -20px підніме елемент на 20 пікселів угору від верхньої частини його відносно розташованого батька.

Хоча відносна позиція приймає властивості зміщення поля, елемент все ще залишається в звичайному або статичному потоці сторінки. У цьому випадку інші елементи не будуть перешкоджати тому, де спочатку був розміщений відносно розташований елемент. Крім того, відносно розміщений елемент може перекривати інші елементи, не зміщуючи їх із положення за замовчуванням.

# Абсолютно розміщені елементи

Теорія: Абсолютно розміщені елементи

Абсолютно розміщені елементи (**position: absolute)** вилучаються зі звичайного потоку документа. Після видалення елемента зі звичайного потоку елементи розташовуються безпосередньо по відношенню до їхнього батьківського елемента, який розташований відносно або абсолютно. Якщо відносно або абсолютно розташованого батьківського елемента немає, абсолютно розташований елемент буде розташовуватися відносно тіла сторінки.

Використання абсолютно розташованих елементів і визначення як вертикального, так і горизонтального зміщення властивостей перемістить елемент із цими значеннями властивостей по відношенню до його відносно розташованого батька. Наприклад, елемент з верхнім значенням 50 пікселів і правим значенням 100 пікселів розташує елемент на 50 пікселів вниз від верхньої частини його відносно розташованого батьківського елемента і на 100 пікселів у правій частині його відносно розташованого батька.

# Фіксоване позиціонування

Теорія: фіксоване позиціонування

Використання **position: fixed** працює так само, як і абсолютне, однак позиціонування є **відносно вікна перегляду** браузера, і воно **не прокручується разом зі сторінкою**. Тим не менш, елементи завжди будуть присутні незалежно від того, де знаходиться користувач на сторінці.

Використання кількох властивостей прямокутного зміщення з фіксованим позиціонуванням призведе до такої ж поведінки, як і абсолютно позиціонований елемент.

Одним із найпоширеніших застосувань фіксованого позиціонування є створення фіксованого верхнього або нижнього колонтитула, прив’язаного до однієї сторони сторінки. Під час прокручування користувачем елемент залишається завжди в області перегляду.

```
.page-header {
   position: fixed;
}
```

# Властивість z-index

Практика: властивість z-index

За своєю природою веб-сторінки вважаються двовимірними, елементи відображаються на осях x і y. Однак, коли ви починаєте розташовувати елементи, вони іноді розміщуються один на одному. Щоб змінити порядок розміщення цих елементів, також відомий як вісь z, слід використовувати властивість **z-index**.

Як правило, елементи розташовуються на осі z так, як вони з'являються в DOM. Елементи у верхній частині DOM, розміщуються позаду елементів, що йдуть після них. Зміна цього стека за допомогою властивості z-index досить проста. Елемент з найвищим значенням z-індексу з'явиться вгорі незалежно від його розташування в DOM.

Щоб застосувати властивість z-індексу до елемента, ви повинні спочатку застосувати значення позиції відносне, абсолютне або фіксоване. Так само, як якщо б ви застосували будь-які властивості коробкового зміщення.

# sticky позиціонування

Практика: sticky

Значення позиції position: sticky є гібрид між відносним і фіксованим положенням. sticky дозволяє елементу діяти так, ніби він відносно розташований, доки його не буде прокручено до певного порогу, після чого він стає фіксованим.

Закріплене позиціонування можна використовувати, наприклад, щоб змусити панель навігації прокручуватися разом зі сторінкою до певної точки, а потім залишатися у верхній частині сторінки.

```
.positioned {
  position: sticky;
  top: 30px;
  left: 30px;
}
```

# Елемент form

Інтерактивні елементи. Атрибути тега input. Мітки елементів. Опис елементів. Перемикачі.

# Форми HTML

Практика: form

Форми HTML використовуються для спілкування з користувачами сайту. Форми дозволяють користувачам вводити дані, які зазвичай надсилаються на веб-сервер для обробки та зберігання або використовуються на стороні клієнта для негайного оновлення інтерфейсу певним чином.

HTML-код веб-форми складається з одного або кількох елементів керування формою, а також деяких додаткових елементів, які допомагають структурувати загальну форму. Елементи керування можуть являти собою одно- чи багаторядкові текстові поля, розкривні списки, кнопки, прапорці або перемикачі, які здебільшого створюються за допомогою елемента <input>.

Форма приймає вхідні дані від відвідувача сайту та зберігає їх у серверній частині сайтк. Серверна частина виконує необхідну обробку переданих даних на основі визначеної бізнес-логіки.

Тег HTML <form> використовується для створення форми HTML і має наступний синтаксис:

```
<form action = "Script URL" method = "GET|POST">
  form elements like input, textarea etc.
  </form>
```

# Елемент form

Практика: Елемент form

Усі форми починаються з елемента <form>, що формально визначає форму:

```
<form action="/my-handling-form-page" method="post">
    ...
</form>
```

Це спеціальний контейнерний елемент для вмісту форм; він також підтримує певні атрибути для налаштування способу поведінки форми. Усі його атрибути є необов’язковими, але стандартною практикою є завжди встановлювати принаймні атрибути дії та методу:

Заборонено вкладати форму в іншу форму. Вкладення може призвести до непередбачуваної поведінки форм.

# Атрибути форми

Практика: атрибути форми

Список найуживаніших атрибутів форми:

1. Action визначає розташування (URL серверного сценарію), куди слід надсилати зібрані дані форми після її надсилання.
2. Method визначає, яким методом HTTP надсилати дані (зазвичай get або post).
3. Target: цільове вікно або фрейм, де буде відображатися результат сценарію. Target приймає такі значення, як _blank, _self, _parent тощо.
4. Enctype: можна використовувати атрибут enctype, щоб вказати, як браузер кодує дані перед тим, як надсилати їх на сервер. Можливі значення:
   - application/x-www-form-urlencoded − стандартний метод, який більшість форм використовує у простих сценаріях.
   - mutlipart/form-data − використовується, коли ви хочете завантажити двійкові дані у формі файлів, наприклад зображення тощо.

# Елемент fieldset

Практика: FORM

Елемент **<fieldset>** - це спосіб створення груп віджетів, які мають однакове призначення, для стилізації та семантичних цілей. Ви можете позначити <fieldset>, включивши елемент **<legend>** під початковим тегом <fieldset>. Текстовий вміст <legend> формально описує призначення <fieldset>, у який він включений.

Багато допоміжних технологій використовуватимуть елемент <legend> так, ніби він є частиною мітки кожного елемента керування всередині відповідного елемента <fieldset>. Наприклад, деякі програми зчитування з екрана озвучують вміст легенди перед тим, як промовляти мітку кожного елемента керування.

```
<fieldset>
   <legend>Tell us about yourself</legend>
     <div class="input-group">
      <label class="form-label" for="name">Your first name *</label>
      <input name="name" id="name" placeholder="Enter your first name">
```

# Елемент label

Практика: form

Елемент **<label>** є формальним способом визначення мітки для віджета форми HTML. Це найважливіший елемент, якщо ви хочете створювати доступні форми - при належній реалізації програми зчитування з екрана озвучуватимуть мітку елемента форми разом із будь-якими відповідними інструкціями, а також це буде корисно для зрячих користувачів.

```
<label class="form-label" for="name">Your first name *</label>
```

Якщо <label> правильно пов’язано з <input> через атрибут **for** (який містить атрибут id елемента <input>), програма зчитування з екрана прочитає `Your first name.`

Існує ще один спосіб пов’язати елемент керування форми з міткою - вкласти елемент керування форми в <label>, неявно пов’язавши його.

```
<label for="name">
   Name: <input type="text" id="name" name="user_name" />
 </label>
```

# Елемент input

Практика: form

Текстові поля **<input>** є найпростішими віджетами форми. Це дуже зручний спосіб дозволити користувачеві вводити будь-які дані. Текстові поля форми HTML - це прості елементи керування введенням звичайного тексту. Це означає, що ви не можете використовувати їх для редагування форматованого тексту. Усі базові елементи керування текстом мають спільну поведінку:

- Їх можна позначити як лише для читання (користувач не може змінити введене значення, але воно все одно надсилається з рештою даних форми) або вимкнено (введене значення не можна змінити та ніколи не надсилається з рештою даних форми) .
- Вони можуть мати заповнювач; це текст, який з'являється в полі введення тексту, який слід використовувати для короткого опису призначення поля.
- Їх можна обмежити розміром і максимальною довжиною (максимальною кількістю символів, які можна ввести в поле).
- Вони можуть отримати користь від перевірки орфографії (за допомогою атрибута spellcheck).

Елемент <input> є унікальним серед елементів HTML, оскільки він може приймати різні форми залежно від значення атрибута type. Він використовується для створення більшості типів віджетів форм, включаючи однорядкові текстові поля, елементи керування часом і датою, елементи керування без введення тексту, такі як прапорці, перемикачі, засоби вибору кольорів і кнопки.

# Однорядкове текстове поле

Практика: form

Однорядкове текстове поле створюється за допомогою елемента <input>, значенням атрибута type якого є text  (текст є значенням за замовчуванням).
Список атрибутів для тегу <input>:

1. **type**: вказує тип вводу. Для тексту буде встановлено **text**.
2. **name**: використовується для надання назви елементу input, який надсилається на сервер для розпізнавання та отримання значення.
3. **value**: використовується для надання початкового значення елементу input.
4. **size**: дозволяє вказати ширину елемента керування в символах.
5. **maxlength**: максимальна кількість символів, які користувач може ввести в текстове поле.

Базовий приклад введення тексту:

```
<form id="contact-form" name="contact-form" action="/mail" method="POST">
        <label for="name" class="">Your name</label>
        <input type="text" id="name" name="name" class="form-control">
```

Щоб визначити значення за замовчуванням для елемента <input>, ви повинні використовувати атрибут value таким чином:<input type="text" value="by default this element is filled with this text" />.

# Елемент textarea

Практика: form

Багаторядкові елементи керування введенням створюються за допомогою HTML-тегу <textarea>.
Список атрибутів для тегу <textarea>.

1. name: Використовується для надання назви елементу керування, який надсилається на сервер для розпізнавання та отримання значення.
2. rows: Вказує кількість рядків поля текстової області.
3. cols: Вказує кількість стовпців поля текстової області

Базовий приклад багаторядкового введення тексту:

```
<form id="contact-form" name="contact-form" action="mail.php" method="POST">
  <div class="row">
        <textarea id="message" name="message" rows="2" class="form-control md-textarea"></textarea>
         <label for="message">Your message</label>
   </div>
```

Якщо ви хочете визначити значення за замовчуванням для <textarea>, ви розміщуєте його між відкриваючим і закриваючим тегами елемента <textarea>.

# Тип password

Практика: form

Значення пароля не додає жодних особливих обмежень до введеного тексту, але воно приховує значення, введене в поле (наприклад, крапками чи зірочками).
Створюються за допомогою HTML-тегу **<input>** з типом password:

```
<form id="contact-form" name="contact-form" action="mail.php" method="POST">
  <div class="row">
    <div class="col-md-6">
      <div class="md-form mb-0">
        <input type="password" id="password" name="password" class="form-control">
        <label for="name" class="">Your password</label>
      </div>
    </div>
```

Атрибут size дозволяє вказати ширину тексту в символах.

Атрибут maxlength дозволяє вказати максимальну кількість символів, які користувач може ввести в текстове поле.

# Прихований тип input

Практика: form

Прихований тип input використовується для створення елемента керування формою, який є невидимим для користувача, але надсилається на сервер разом із рештою даних форми після надсилання. Оскільки воно приховане, користувач не може бачити чи навмисно редагувати значення, воно ніколи не отримає фокус, і програма зчитування з екрана також не помітить його.

```
<input type="hidden" id="timestamp" name="timestamp" value="1286705410" />
```

Якщо ви створюєте такий елемент, необхідно задати його ім'я та атрибути значення. Значення можна динамічно встановлювати за допомогою JavaScript. Прихований тип введення не повинен мати пов’язаної мітки.

# Поле електронної адреси

Практика: form

Поле електронної адреси встановлюється за допомогою значення **email** для атрибута type:

```
<label class="form-label" for="email">Your email *</label>
<input class="form-control" type="email" name="email" id="email" placeholder="Enter your  email" required="required">
```

Якщо використовується цей тип, користувач повинен ввести дійсну адресу електронної пошти в поле. Будь-який інший вміст викликає у браузері повідомлення про помилку під час надсилання форми.

Ви також можете використовувати атрибут multiple в поєднанні з типом введення електронної пошти, щоб дозволити вводити кілька адрес електронної пошти в одному введенні (розділених комами):

```
<input type="email" id="email" name="email" multiple />
```

На деяких пристроях - зокрема на сенсорних пристроях із динамічною клавіатурою, наприклад на смартфонах - може бути представлена віртуальна клавіатура, яка більше підходить для введення адреси електронної пошти, включаючи клавішу @.

# Button

Практика: form

Елемент **<button>** може приймати атрибут тип **submit, reset** або **button**.

Кнопки завжди поводяться однаково незалежно від того, використовуєте ви елемент <button> або <input>.

Основна перевага елемента <button> полягає в тому, що елемент <input> допускає лише звичайний текст у своїй мітці, тоді як елемент <button> допускає повний вміст HTML, дозволяючи більш складний, творчий вміст кнопки.

# Прапорець

Практика: form

Прапорець створюється за допомогою елемента <input> з атрибутом type = checkbox. Будь-які прапорці з атрибутом checked під час завантаження відповідають псевдокласу :default, навіть якщо вони не позначені. Прапорці використовуються, коли потрібно вибрати більше одного параметра. Список атрибутів для тегу <checkbox>.

1. type вказує тип керування.
2. name використовується для надання назви елементу керування, який надсилається на сервер для розпізнавання та отримання значення.
3. value - значення, яке використовуватиметься, якщо встановлено прапорець.
4. checked встановює прапорець, якщо ви хочете вибрати його за замовчуванням.

Приклад HTML-коду для форми з двома прапорцями:

```
<form>
 <input type = "checkbox" name = "maths" value = "on"> Maths
 <input type = "checkbox" name = "physics" value = "on"> Physics
</form>
```

# Радіо-кнопка

Практика: form

Перемикач створюється за допомогою елемента <input> з атрибутом type = radio. Перемикачі використовуються, коли з багатьох варіантів потрібно вибрати лише один.

Список атрибутів перемикача.

1. type вказує тип керування.
2. name використовується для надання назви елементу керування, який надсилається на сервер для розпізнавання та отримання значення.
3. value - це значення, яке використовуватиметься, якщо вибрано перемикач.
4. checked встановлює прапорець, якщо ви хочете вибрати його за замовчуванням.

```
<form>
 <input type = "radio" name = "subject" value = "maths"> Maths
 <input type = "radio" name = "subject" value = "physics"> Physics
</form>
```

# Поле пошуку

Практика: form

Поля пошуку призначені для створення вікон пошуку на сторінках і в програмах:

```
<input type="search" id="search" name="search" />
```

Основна відмінність між текстовим полем і полем пошуку полягає в тому, як браузер стилізує його вигляд. Часто поля пошуку відображаються із закругленими кутами; вони також іноді відображають «ⓧ», який очищає поле від будь-яких значень після натискання. Крім того, на пристроях із динамічною клавіатурою клавіша введення на клавіатурі може читати «пошук» або відображати піктограму збільшувального скла.

Значення поля пошуку можна автоматично зберігати та повторно використовувати для автоматичного заповнення на кількох сторінках одного веб-сайту; у більшості сучасних браузерів це відбувається автоматично.

# Поле номера телефону

Практика: form

Поле номера телефону - спеціальне поле для заповнення номерів телефонів

```
<label class="form-label" for="phone">Your phone number</label>
<input class="form-control" type="tel" name="phone" id="phone" placeholder="Enter your phone number">
```

Якщо отримати доступ через сенсорний пристрій із динамічною клавіатурою, більшість пристроїв відображатиме цифрову клавіатуру, коли зустрічається type="tel", тобто цей тип корисний, коли потрібна цифрова клавіатура, а не лише для телефону чисел.

Завдяки широкому розмаїттю форматів телефонних номерів у всьому світі цей тип поля не накладає жодних обмежень на значення, введене користувачем (це означає, що воно може містити літери тощо).

# Поле URL-адрес

Практика: form

Поле URL - спеціальний тип поля для введення URL-адрес:

```
<input type="url" id="url" name="url" />
```

Браузер повідомить про помилку, якщо не введено протокол (наприклад, http:) або якщо URL-адреса має неправильний формат. На пристроях із динамічною клавіатурою стандартна клавіатура часто відображатиме частину або всю двокрапку, крапку та косу риску як клавіші за замовчуванням.

# Числове поле

Практика: form

Елементи керування для введення чисел можна створити за допомогою типу type="number". Цей елемент керування виглядає як текстове поле, але дозволяє вводити лише числа з плаваючою комою та зазвичай містить кнопки у формі обертової кнопки для збільшення та зменшення значення елемента керування. На пристроях із динамічною клавіатурою зазвичай відображається цифрова клавіатура.

За допомогою типу введення числа ви можете обмежити мінімальні та максимальні дозволені значення, встановивши атрибути min та max.
Ви також можете використовувати атрибут step, щоб встановити збільшення та зменшення кроку, спричинене натисканням кнопок обертання. За замовчуванням тип введення числа перевіряє, лише якщо число є цілим. Щоб дозволити числа з плаваючою фігурою, вкажіть step="any". Якщо опущено, значення кроку за замовчуванням дорівнює 1, тобто дійсні лише цілі числа.

```
<input type="number" name="change" id="pennies" min="0" max="1" step="0.01" />
```

# Select Box

Практика: form

Поле вибору, також називається розкривним списком.

```
<select name = "dropdown">
   <option value = "Maths" selected>Maths</option>
   <option value = "Physics">Physics</option>
</select>
```

Список важливих атрибутів тегу <select>

1. name використовується для надання назви елементу select, що надсилається на сервер для розпізнавання та отримання значення.
2. size можна використати для представлення списку, що прокручується.
3. multiple - якщо встановлено, то користувачеві дозволяється вибирати кілька пунктів у меню.

Список важливих атрибутів тегу <option>

1. value - значення, яке використовуватиметься, якщо вибрано параметр у полі вибору.
2. selected вказує, що цей параметр має бути вибраним під час завантаження сторінки.

# Псевдокласи UI

Практика: form

Деякі псевдокласи інтерфейсу користувача, які зазвичай використовуються з елементами форми, можна використовувати для показу сповіщень на основі атрибутів елемента:

**:focus, :required, :valid, :invalid,:checked**

Псевдоклас :focus використовується для стилізації елемента, який отримав фокус за допомогою вказівного пристрою, дотику на сенсорному пристрої або за допомогою клавіатури.

```
a:focus {
  color: green;
}
Or:
input:focus {
  background: #eee;
}
```

Псевдоклас CSS **:in-range** представляє елемент <input>, поточне значення якого знаходиться в межах діапазону, визначеного атрибутами min і max.

Цей псевдоклас корисний для надання користувачеві візуальної індикації того, що поточне значення поля знаходиться в межах дозволених меж.

Цей псевдоклас застосовується лише до елементів, які мають обмеження діапазону. За відсутності такого обмеження елемент не може бути ні «в діапазоні», ні «поза діапазоном».

```
input:in-range {
   background-color: rgb(0 255 0 / 0.25);
 }
```

# Фокусування на елементах форми

Практика: form

У деяких браузерах під час фокусування на елементах форми з'являється синій контур. Ви можете використати псевдоклас :focus, щоб видалити його та додати більше ефектів, коли елемент у фокусі.

Наступний код видаляє контур фокуса для всіх елементів.

```
*:focus {outline:none !important}
```

Щоб додати контур фокуса:

```
input[type=text]:focus {
 background-color: #ffd969;
 border-color: #000;
 // and any other style
}
```

# :require & :placeholder-shown

Практика: form

Оскільки ми не можемо використовувати псевдоелементи для елемента введення, тому можна додати додатковий div після елемента input, щоб відобразити повідомлення.

```
<label><span>Name</span>
   <input type="text" name="name" placeholder="your name" required /><div></div>
 </label>
```

Тоді ми можемо використати :required, щоб додати певний стиль для input, яке має бути заповненим.

```
input:required + div::before {
   content: "(required)";
   color: red;
 }
```

Якщо ми хочемо приховати попередження після того, як користувач заповнить введені дані, ми можемо використати :placeholder-shown, щоб зробити цей трюк.

```
input:required + div::before {
   display: none;
 }
 input:placeholder-shown + div::before {
   display: block;
 }
```

# Повідомлення про помилку

Практика: form

Якщо у вмісті, який надає користувач, є якась помилка, ми можемо захотіти повідомити користувача, що форму ще не заповнено. Тут ми можемо застосувати :invalid для відображення результату.

```
<form>
 <label>
   <span>Name</span>
   <input type="text" name="name" placeholder="your name" required />
   <div></div>
 </label>
 <button type="submit"></button>
</form>
button:before {
 content: "Ready to Submit";
}
form:invalid > button {
 background: darkgray;
}
form:invalid > button:before {
 content: "Not Ready";
}
```

# Параметр за замовчуванням

Практика: form

Іноді ми встановлюємо параметр за замовчуванням для прапорця або радіо. Добре повідомити користувачеві, що цей параметр позначено за замовчуванням. Ми можемо використовувати :default, щоб додати для цього додатковий стиль.

```
 <span>Fields</span>
 <label>
   <input type="checkbox" value="fe" checked />
   Front-End Developer
   <div></div>
 </label>
 <label>
   <input type="checkbox" value="be" />
   Back-End Developer
   <div></div>
 </label>
input:default + div::before {
 content: "(default)";
 color: red;
}
```

# Заборона редагування

Практика: form

Коли користувач редагує форму, ми можемо не дозволити йому редагувати певні стовпці (наприклад, ідентифікатор користувача). Тож ми можемо використовувати :read-only display інший стиль, щоб ідентифікувати це.

```
<label>
 <span>Referred by</span>
 <input type="text" value="oahehc@gamil.com" readonly />
 <div></div>
</label>
input:read-only + div::before {
 content: "(read-only)";
 color: red;
}
```

# Значення виходить за межі

Практика: form

Псевдоклас CSS **:in-range** представляє елемент <input>, поточне значення якого знаходиться в межах діапазону, визначеного атрибутами min і max. Цей псевдоклас корисний для надання користувачеві візуальної індикації того, що поточне значення поля знаходиться в дозволених межах.

Цей псевдоклас застосовується лише до елементів, які мають обмеження діапазону. За відсутності такого обмеження елемент не може бути ні «в діапазоні», ні «поза діапазоном».

```
input:in-range { background-color: rgb(0 255 0 / 0.25); }
```

Якщо ми встановили діапазон для введення, ми можемо використовувати :out-of-range, щоб попередити користувача, коли вхідне значення виходить за межі.

```
<label><span>Years of Experience</span>
 <input type="number" min="0" max="100" />
 <div></div>
</label>
input:out-of-range + div::before {
 content: "(out-of-range)";
 color: red;
}
```

# ВПРАВИ

Вправи до цього заняття знаходяться за посиланням
https://github.com/coachjanus/frontend-dev

# Посилання на довідники



Vs Code : https://code.visualstudio.com/

Webstorm: https://www.jetbrains.com/webstorm/

Web technology For developers https://developer.mozilla.org/en-US/docs/Web

CSS: https://developer.mozilla.org/en-US/docs/Web/CSS

Консорціум Всесвітнього павутиння https://www.w3.org/

# Дякую за увагу!

# До наступної зустрічі!

QALight

www.qalight.ua
info@qalight.ua